

JavaScript Advanced Question Answer

1. What is JavaScript, and how is it different from Java?

JavaScript is a scripting language primarily used for web development.

Differences:

- JavaScript is dynamically typed; Java is statically typed.
 - JavaScript uses prototypes; Java uses classes.
 - JavaScript runs in browsers and Node.js; Java runs on the JVM.
-

2. What are the different data types in JavaScript?

Primitive Types

- String
- Number
- Boolean
- Undefined
- Null
- Symbol
- BigInt

Non-Primitive Types

- Object
 - Array
 - Function
-

3. What is the difference between null and undefined?

null

undefined

Intentional empty value Variable declared but not assigned

Type: object

Type: undefined

```
let a;  
console.log(a); // undefined
```

```
let b = null;
```

4. What is the difference between == and ===?

== performs type coercion.

```
5 == "5" // true
```

=== checks value and type.

```
5 === "5" // false
```

5. What are truthy and falsy values?

Falsy Values

false

0

""

null

undefined

NaN

Everything else is truthy.

6. What is type coercion?

Automatic conversion from one type to another.

```
"5" + 2
```

```
// "52"
```

7. What is NaN? How can you check it?

NaN means "Not a Number".

```
Number("abc")
```

Check:

```
Number.isNaN(value)
```

8. What is the difference between let, const, and var?

Feature	var	let	const
---------	-----	-----	-------

Scope	Function	Block	Block
-------	----------	-------	-------

Redeclare	Yes	No	No
-----------	-----	----	----

Feature	var	let	const
Reassign	Yes	Yes	No

9. What is variable hoisting?

Declarations are moved to the top during execution.

```
console.log(a);
```

```
var a = 10;
```

Output:

undefined

10. What is the Temporal Dead Zone (TDZ)?

The period between entering scope and initializing a let or const variable.

```
console.log(a);
```

```
let a = 5;
```

Throws ReferenceError.

11. Function Declaration vs Function Expression

Declaration:

```
function greet(){}
```

Expression:

```
const greet = function(){};
```

Declarations are hoisted.

12. What are Arrow Functions?

Short syntax for functions.

```
const add = (a,b) => a+b;
```

They don't have their own this.

13. What is the this keyword?

Refers to the object that invokes the function.

```
const user = {  
  name:"John",  
  show(){  
    console.log(this.name);  
  }  
};
```

14. How does this behave inside an Arrow Function?

Arrow functions inherit this from their surrounding scope.

```
const obj = {  
  name:"John",  
  show: () => console.log(this)  
};
```

15. What is a Callback Function?

A function passed as an argument to another function.

```
function greet(cb){  
  cb();  
}
```

16. What are Higher-Order Functions?

Functions that:

- Accept functions as arguments
- Return functions

Examples:

```
map()  
filter()  
reduce()
```

17. What is a Closure?

A function that remembers variables from its outer scope.

```
function outer(){  
  let count = 0;  
  
  return function(){  
    count++;  
    return count;  
  }  
}
```

```
};  
}
```

18. What is Currying?

Transforming a function with multiple arguments into nested functions.

```
function add(a){  
  return function(b){  
    return a+b;  
  };  
}
```

```
add(2)(3);
```

19. What is Function Borrowing?

Using a method of one object on another object.

```
person1.show.call(person2);
```

20. What are IIFEs?

Immediately Invoked Function Expressions.

```
(function(){  
  console.log("Hello");  
})();
```

21. What is Lexical Scope?

A function can access variables defined in its outer scope.

```
function outer(){  
  let x = 10;  
  
  function inner(){  
    console.log(x);  
  }  
}
```

22. What is the Scope Chain?

JavaScript searches variables:

1. Local scope

2. Parent scope
 3. Global scope
-

23. What is Execution Context?

Environment in which JavaScript code executes.

Contains:

- Variables
 - Functions
 - this
-

24. What is the Call Stack?

A stack that tracks function execution.

a()
→ b()
→ c()

25. Global Scope, Function Scope, and Block Scope

```
var a = 1; // Global
```

```
function test(){  
  let b = 2; // Function  
}
```

```
{  
  let c = 3; // Block  
}
```

26. Primitive vs Reference Data Types

Primitive

Copied by value.

Number
String
Boolean

Reference

Copied by reference.

Object
Array
Function

27. How do you create an Object?

```
const user = {  
  name:"John"  
};
```

28. What is Object Destructuring?

Extract properties into variables.

```
const user = {name:"John"};
```

```
const {name} = user;
```

29. What is the Spread Operator (...)?

Expands elements.

```
const arr2 = [...arr1];
```

30. What is the Rest Parameter?

Collects arguments into an array.

```
function sum(...nums){  
  return nums;  
}
```

31. Difference between map(), filter(), and reduce()

map()

Transforms elements.

```
arr.map(x => x*2);
```

filter()

Filters elements.

```
arr.filter(x => x>5);
```

reduce()

Produces a single value.

```
arr.reduce((a,b)=>a+b);
```

32. Difference between `forEach()` and `map()`

<code>forEach</code>	<code>map</code>
----------------------	------------------

No return value	Returns new array
-----------------	-------------------

Used for iteration	Used for transformation
--------------------	-------------------------

33. Shallow Copy vs Deep Copy

Shallow Copy

```
const copy = {...obj};
```

Nested objects still share references.

Deep Copy

```
structuredClone(obj);
```

Everything copied independently.

34. What are Object Methods?

Functions inside objects.

```
const user = {  
  greet(){  
    console.log("Hello");  
  }  
};
```

35. How can you iterate through object properties?

```
for(let key in obj){  
  console.log(key);  
}
```

or

```
Object.keys(obj);
```

36. Synchronous vs Asynchronous Programming

Synchronous

Tasks execute one after another.

Asynchronous

Tasks execute without blocking.

37. What is the Event Loop?

The Event Loop moves callbacks from queues to the call stack when the stack becomes empty.

38. What are Promises?

Objects representing future completion or failure of async operations.

```
const p = new Promise((resolve,reject)=>{  
  resolve("Success");  
});
```

39. What are the states of a Promise?

1. Pending
 2. Fulfilled
 3. Rejected
-

40. Callback vs Promise

Callback

```
getData(cb);
```

Promise

```
getData()  
.then()  
.catch();
```

Promises avoid callback hell.

41. What is async/await?

A cleaner way to work with Promises.

```
async function getData(){  
  const data = await fetch(url);  
}
```

42. How does Promise.all() work?

Runs multiple promises in parallel.

```
Promise.all([p1,p2,p3]);
```

Fails if any promise rejects.

43. Difference between Promise.all() and Promise.allSettled()

Promise.all()

Fails immediately if one promise rejects.

Promise.allSettled()

Waits for all promises regardless of success or failure.

44. What are Microtasks and Macrotasks?

Microtasks

- Promise.then()
- queueMicrotask()

Macrotasks

- setTimeout()
- setInterval()

Microtasks execute before macrotasks.

45. Output of the following code?

```
console.log("1");
```

```
setTimeout(() => console.log("2"), 0);
```

```
Promise.resolve().then(() => console.log("3"));
```

```
console.log("4");
```

Output:

```
1
4
3
2
```

Because:

- Synchronous code runs first.

- Promise (microtask) runs next.
 - setTimeout (macrotask) runs last.
-

46. What is Prototypal Inheritance?

Objects can inherit properties and methods from other objects through prototypes.

```
const person = {  
  greet(){  
    console.log("Hello");  
  }  
};
```

```
const user = Object.create(person);
```

47. What is the Prototype Chain?

JavaScript searches properties:

1. Object itself
 2. Prototype
 3. Parent prototype
 4. Object.prototype
 5. null
-

48. What are ES6 Modules?

Used to split code into reusable files.

Export

```
export const name = "John";
```

Import

```
import { name } from "./file.js";
```

49. What is Event Delegation?

Attaching a single event listener to a parent element to handle child events.

```
document  
  .getElementById("list")  
  .addEventListener("click", (e) => {
```

```
    console.log(e.target);  
  });
```

Improves performance.

50. What is the difference between Debounce and Throttle?

Debounce

Executes after the user stops triggering events.

Example:

- Search input

Throttle

Executes at fixed intervals.

Example:

- Scroll events

Debounce	Throttle
Delays execution	Limits execution rate
Useful for search boxes	Useful for scrolling